

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Full Stack development MERN Practical
Practical 8 – Mini Project

Roll No.:T009	Name: Siddhesh Kamble
Class:TYIT	Batch:1
Date of Assignment: 26-9-2026	Date/Time of Submission: 26-9-2026

Project Title: College Canteen Pre-Order System

Aim :

To develop a web-based College Canteen Pre-Order System using React, Axios, Node.js, Express.js and MongoDB that allows students to view food items, place orders and view stored orders through REST APIs.

Project Requirements :

- View available food items and their prices.
- Enter student details such as name, roll number and course.
- Select a food item and quantity.
- Place an order using a POST API.
- Store order details in MongoDB.
- Retrieve stored orders using a GET API.
- Display food items and placed orders on the React page.

Technologies Used :

- React
- Axios
- Node.js
- Express.js
- MongoDB

API Endpoints :

Method	Endpoint	Purpose
GET	/food	Retrieve available food items.
POST	/orders	Add a new food order.
GET	/orders	Retrieve placed orders.

MongoDB Document Structure:

Database: college | Collection: students

Field	Data Type
studentName	String
rollNo	Number
course	String
foodItem	String
price	Number
quantity	Number
total	Number

Project Setup:

1. Create a project folder named CanteenSystem.
2. Create a backend folder and open it in VS Code.
3. Run npm init -y.
4. Install the required packages using npm install express mongoose cors.
5. Create server.js and paste the backend code given below.
6. Create a React application using Vite, or use an existing React project.
7. Install Axios using npm install axios.
8. Replace App.jsx and App.css with the code given below.
9. Start MongoDB Server.
10. Run the backend using node server.js. The server runs on port 5000.
11. Run the React application using npm run dev.
12. Open the React URL shown in the terminal, usually http://localhost:5173.

Backend Code:

server.js

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
```

```
const app = express();

app.use(cors());
app.use(express.json());

mongoose.connect("mongodb://127.0.0.1:27017/college")
  .then(() => console.log("MongoDB Connected"));

const Order = mongoose.model("students", new mongoose.Schema( {
  studentName: String,
  rollNo: Number,
  course: String,
  foodItem: String,
  price: Number,
  quantity: Number,
  total: Number
}));

const food = [
  { name: "Pizza", price: 100 },
  { name: "Veg Burger", price: 60 },
  { name: "Sandwich", price: 50 },
  { name: "Samosa", price: 20 }
];

// GET /food
app.get("/food", (req, res) => {
  res.json(food);
});

// POST /orders
app.post("/orders", async (req, res) => {
  const { studentName, rollNo, course, foodItem, quantity } = req.body;

  const item = food.find(f => f.name === foodItem);
  const total = item.price * quantity;

  const order = new Order({
    studentName,
    rollNo,
    course,
    foodItem,
    price: item.price,
    quantity,
    total
  });

  await order.save();
  res.json(order);
});

// GET /orders
app.get("/orders", async (req, res) => {
  const orders = await Order.find();
  res.json(orders);
});

app.listen(5000, () => console.log("Server running on 5000"));
```

Frontend Code :

App.jsx

```
import { useEffect, useState } from "react";
import axios from "axios";
import "./App.css";

function App() {
  const [food, setFood] = useState([]);
  const [orders, setOrders] = useState([]);

  const [form, setForm] = useState( {
    studentName: "",
    rollNo: "",
    course: "",
    foodItem: "",
    quantity: 1
  });

  useEffect(() => {
    axios.get("http://localhost:5000/food")
      .then(res => setFood(res.data));

    getOrders();
  }, []);

  const getOrders = () => {
    axios.get("http://localhost:5000/orders")
      .then(res => setOrders(res.data));
  };

  const change = e => {
    setForm({ ...form, [e.target.name]: e.target.value });
  };
}
```

```
};

const order = e => {
  e.preventDefault();

  axios.post("http://localhost:5000/orders", {
    ...form,
    rollNo: Number(form.rollNo),
    quantity: Number(form.quantity)
  }).then(() => {
    alert("Order Placed!");
    getOrders();

    setForm({
      studentName: "",
      rollNo: "",
      course: "",
      foodItem: "",
      quantity: 1
    });
  });
};

return (
  <div className="container">
    <h1>College Canteen Pre-Order System</h1>

    <h2>Food Items</h2>

    <div className="food">
      {food.map(f => (
        <div key={f.name}>
          <b>{f.name}</b>
          <p>₹ {f.price}</p>
        </div>
      ))}
    </div>

    <h2>Student Order Form</h2>

    <form onSubmit={order}>
      <input
        name="studentName"
        placeholder="Student Name"
        value={form.studentName}
        onChange={change}
        required
      />

      <input
        name="rollNo"
        placeholder="Roll No"
        value={form.rollNo}
        onChange={change}
        required
      />

      <input
        name="course"
        placeholder="Course"
        value={form.course}
        onChange={change}
        required
      />

      <select
        name="foodItem"
        value={form.foodItem}
        onChange={change}
        required
      >
        <option value="">Select Food</option>
        {food.map(f => (
          <option key={f.name} value={f.name}>
            {f.name} - ₹ {f.price}
          </option>
        ))}
      </select>

      <input
        type="number"
        name="quantity"
        min="1"
        value={form.quantity}
        onChange={change}
      />

      <button>Place Order</button>
    </form>

    <h2>Placed Orders</h2>
```

```
<table>
<thead>
  <tr>
    <th>Name</th>
    <th>Roll No</th>
    <th>Course</th>
    <th>Food</th>
    <th>Price</th>
    <th>Qty</th>
    <th>Total</th>
  </tr>
</thead>

<tbody>
  {orders.map(o => (
    <tr key={o._id}>
      <td>{o.studentName}</td>
      <td>{o.rollNo}</td>
      <td>{o.course}</td>
      <td>{o.foodItem}</td>
      <td>₹{o.price}</td>
      <td>{o.quantity}</td>
      <td>₹{o.total}</td>
    </tr>
  ))}
</tbody>
</table>
</div>
);
}
```

export default App;

App.css

```
body {
  font-family: Arial;
  background: #f2f2f2;
}

.container {
  width: 90%;
  margin: auto;
}

h1 {
  text-align: center;
  color: #333;
}

.food {
  display: flex;
  gap: 15px;
}

.food div {
  background: white;
  padding: 15px;
  width: 150px;
  border-radius: 8px;
}

form {
  background: white;
  padding: 20px;
  margin: 20px 0;
}

input, select, button {
  display: block;
  width: 100%;
  padding: 10px;
  margin: 8px 0;
  box-sizing: border-box;
}

button {
  background: green;
  color: white;
  border: none;
  cursor: pointer;
}

table {
  width: 100%;
  background: white;
  border-collapse: collapse;
}

th, td {
  padding: 10px;
  border: 1px solid #ccc;
  text-align: center;
```

```

}
th {
  background: #333;
  color: white;
}

```

Running the Project :

1. Start MongoDB Server.
2. Open the backend terminal and run: node server.js
3. Check the message: MongoDB Connected and Server running on 5000.
4. Open another terminal in the React project folder.
5. Run: npm run dev
6. Open the local React URL in the browser.
7. View the food items, enter student details, select food and quantity, and click Place Order.
8. The order is sent to POST /orders and stored in MongoDB.
9. The placed orders are retrieved using GET /orders and displayed in the table.

Expected Output :

- Food Item List
- Student Order Form
- Place Order button
- Placed Order List/Table

Course						
Select Food						
1						
Place Order						

Name	Roll No	Course	Food	Price	Qty	Total
Siddhesh	13	B.Sc. IT	Pizza	₹100	2	₹200
Soham	5	B.Sc. IT	Veg Burger	₹60	1	₹60
Pranay	16	B.Sc. IT	Sandwich	₹50	3	₹150
Sagar	14	B.Sc. IT	Samosa	₹20	4	₹80
Raj	16	CS	Cold Coffee	₹70	2	₹140
Som	4	IT	Pizza	₹100	1	₹100
Sameer	12	B.Sc. IT	Veg Burger	₹60	2	₹120

Result :

The College Canteen Pre-Order System was successfully developed using React, Axios, Node.js, Express.js and MongoDB. The application displays food items, accepts student orders, stores the order details in MongoDB through a POST API, and retrieves and displays placed orders using a GET API.